Documento de Visão – P4 Doctors

Versão 1.1

Revisões do Documento

Revisões são melhoramentos na estrutura do documento e também no seu conteúdo. O objetivo primário desta tabela é a fácil identificação da versão do documento. Toda modificação no documento deve constar nesta tabela.

Versão	Data	Autores	Observações
1.0	13/03/2026	Pablo Henrique Cardoso Ramos, Ruan Alves de Oliveira, Said Salles Paiva	Primeira versão do documento de visão, focando na construção dos seguintes tópicos: Documento de Visão do Sistema, Identificação dos Stakeholders, Definição do Escopo do Sistema, Levantamento inicial das Necessidades/Problemas que o sistema pretende atender
1.1	25/03/2026	Pablo Henrique Cardoso Ramos, Ruan Alves de Oliveira, Said Salles Paiva	Segunda versão do documento de visão, essa versão apresenta o preenchimento de novos tópicos como Requisitos Não Funcionais do Produto, Restrições Técnicas e Dicionário de Dados.
1.2	08/04/2026	Pablo Henrique Cardoso Ramos, Ruan Alves de Oliveira, Said Salles Paiva	Terceira versão do documento de visão, focado no aprimoramento e clareza dos requisitos.

Auditorias do Documento

Auditorias são inspeções formais conduzidas pela equipe de Qualidade e Conformidade da Empresa, e tem por objetivo garantir a qualidade dos artefatos gerados durante o processo de desenvolvimento do mesmo. Toda modificação no documento deve constar nesta tabela.

Versão	Data	Autores	E-mail	Observações
1.0	13/03/2026	Pablo Henrique Cardoso Ramos, Ruan Alves de Oliveira, Said Salles Paiva		Primeira versão do documento de visão, focando na construção dos seguintes tópicos: Documento de Visão do Sistema, Identificação dos Stakeholders, Definição do Escopo do Sistema, Levantamento inicial das Necessidades/Problemas que o sistema pretende atender

Sumário

1. Introdução 4
   1.1 Finalidade 4
   1.2 Escopo 4
   1.3 Definições, Acrônimos e Abreviações 4
   1.4 Referências 4
      1.4.1 Especificações Suplementares 4

2. Contextualização 5
   2.1 Descrição do Problema 5
   2.2 Sentença de Posição do Produto 5

3. Descrição dos *Stakeholders* e dos Usuários 5
   3.1 Principais *Stakeholders* e Usuários 5
   3.2 Necessidades Chave dos *Stakeholders* e dos Usuários 6

4. Visão Geral do Produto 6
   4.1 Perspectiva do Produto 6
   4.2 Estimativa de Custo Inicial 7
   4.3 Suposições e dependências 7
   4.4 Premissas e Dependências 7

5. Precedência e Prioridades 7

6. Requisitos Não-Funcionais do Produto 9

7. Restrições Técnicas 9
   7.1 Padrões Aplicáveis 9
   7.2 Restrições Aplicáveis 10

8. Manual de Arquitetura 10
   8.1 Visão de Implementação 10
   8.2 Mecanismo Arquiteturais 10

9. Dicionário de Dados 11
   9.1 Finalidade 11
   9.2 Tabela: atendimentos 11
   9.3 Tabela: pessoas 12
   9.4 Tabela: médicos 13
   9.5 Tabela: pacientes 13
   9.6 Tabela: equipe_apoio 13
   9.7 Tabela: vinculo_equipe_medico 14
   9.8 Tabela: agendamentos 14

Documento de Visão – P4 Doctors

1. Introdução

1.1 Finalidade

A finalidade deste documento é coletar, analisar e definir as necessidades de alto nível e as características do sistema P4 Doctors. Focando nas capacidades requeridas pelos stakeholders e nos motivos que fundamentam essas necessidades. Este documento servirá como base de alinhamento para o desenvolvimento da primeira versão (MPV) da plataforma, garantindo que o foco permaneça na substituição de controles manuais por um sistema estruturado e analítico.

1.2 Escopo

Nesta versão 1.0, o escopo do P4 Doctors compreende a construção de uma aplicação web desenvolvida em Python (utilizando o framework Streamlit, Plotly) e integrada a um banco de dados (como PostgreSQL (Supabase).

O sistema contemplará:

- Uma interface gráfica web intuitiva;
- Sistema de autenticação;
- Módulo de Gestão (CRUD Completo) para lançamento de consultas médicas (paciente, local, tipo de consulta, valor, e relatório clínico);
- Módulo Analítico contendo dashboards para visualização de métricas e produtividade do consultório.

1.3 Definições, Acrônimos e Abreviações

- Mais informações serão encontradas no Glossário do Sistema.

1.4 Referências

1.4.1 Especificações Suplementares

- Documento de Regra de Negócio
- Documento de Requisitos
- Documento Manual do Usuário
- Documento Dicionário de Dados

2. Contextualização

2.1 Descrição do Problema

Problemas	• Registro manual, descentralizado e não padronizado de atendimentos clínicos (como o uso de planilhas genéricas). • Falta de confiabilidade e segurança de ao se utilizar uma planilha.
Pessoas Atingidas	• Médicos, profissionais de saúde e secretárias de clínicas e consultórios particulares.
Cujo impacto é	• O alto risco de perda do histórico de pacientes; • Dificuldade na extração rápida de métricas de produtividade e faturamento; • Poluição visual na leitura de relatórios; • Gasto excessivo de tempo em tarefas administrativas repetitivas.
Uma solução bem-sucedida traria	• A centralização dos dados médicos em um ambiente seguro, ágil e estruturado em nuvem permitindo a geração automática de dashboards; • Facilidade e agilidade na recuperação do histórico de consultas através de buscas otimizadas.

2.2 Sentença de Posição do Produto

Para	Médicos e profissionais de saúde
O	P4 Doctors é um sistema web de gestão de consultas e painel analítico
Que	Necessitam de uma ferramenta centralizada para registrar seus atendimentos clínicos diários e visualizar métricas de produtividade e retorno financeiro de forma automatizada
Diferente de	Planilhas genéricas (como Excel e Google Sheets) e prontuários em papel
Nosso produto	Oferece uma interface visual intuitiva focada especificamente na rotina médica, eliminando a necessidade de criar fórmulas manuais e fornecendo dashboards instantâneos que facilitam a gestão do tempo, faturamento e do consultório.

3. Descrição dos *Stakeholders* e dos Usuários

3.1 Principais *Stakeholders* e Usuários

Identificação	Responsabilidades	*Stakeholders*
Médico	Alimentar o sistema com os relatórios das consultas e valores dos plantões/atendimentos,	Usuário Final Primário/Cliente

	analisar os valores de produtividade e faturamento e para a gestão pessoal e consultar o histórico de pacientes para acompanhamento clínico.	
Secretária/Recepcionista	Auxiliar no cadastro inicial dos pacientes, realizar buscas no histórico de atendimentos e inserir dados administrativos, quando aplicável.	Usuário Operacional
Desenvolvedor	Desenhar a arquitetura da aplicação em Python, configurar a interface no Streamlit, estruturar o banco de dados e garantir a integridade e disponibilidade do sistema.	Equipe Técnica/Mantenedor

3.2 Necessidades Chave dos *Stakeholders* e dos Usuários

No.	Descrição	Prioridade do Cliente (Crítico, Útil e Importante)	Observações
1	Registrar rapidamente os dados essenciais da consulta (Nome do Paciente, Local, Tipo, Valor e Relatório Clínico).	Crítico	Necessidade de um formulário limpo e direto (Create do CRUD).
2	Visualizar métricas de produtividade e faturamento (total de consultas no mês, valores, atendimentos por local e por tipo).	Crítico	O Dashboard deve ser a primeira tela a ser carregada ao entrar no sistema.
3	Consultar e buscar o histórico de um paciente específico de forma ágil.	Importante	Tabela de listagem de registros com barra de pesquisa e filtros.
4	Editar ou excluir um registro lançado com informações incorretas.	Importante	Funções de Update e Delete para manter a base de dados limpa.
5	Acessar a plataforma através de um navegador web, sem necessidade de instalar programas complexos.	Útil	A escolha do Streamlit atende perfeitamente a essa necessidade.

4. Visão Geral do Produto

4.1 Perspectiva do Produto

A tabela abaixo correlaciona as dores operacionais de uma clínica médica com as soluções tecnológicas implementadas no sistema:

Necessidades	Funcionalidades Correspondentes
1. Centralizar e organizar o histórico clínico dos pacientes.	1.1 – Cadastro de Pacientes e Prontuário Digital.

2. Eliminar conflitos de agenda e otimizar o tempo dos médicos.	2.1 Gestão de Agenda Multi-profissional com bloqueios de horários.
3. Monitorar a receita gerada por consultas e procedimentos.	3.1 Registro de Faturamento Clínico.
4. Visualizar indicadores de desempenho e saúde financeira.	4.1 Dashboards Analíticos e Financeiros.
5. Assegurar a integridade e o sigilo dos dados dos pacientes.	5.1 Autenticação Segura (Bcrypt) e Controle de Acesso (RBAC).

4.2 Estimativa de Custo Inicial

Para um projeto baseado em arquitetura Cloud-native (PaaS e DBaaS), o custo inicial de infraestrutura é extremamente baixo, permitindo uma fase de validação sem grandes investimentos em hardware. Tendo um custo inicial apenas para obter um domínio próprio para a aplicação de aproximadamente R$ 88,39. Quanto ao database e o hosting a aplicação usaria planos gratuitos, podendo no futuro migrar para uma solução paga.

4.3 Suposições e dependências

Esta seção define os requisitos mínimos necessários para o desenvolvimento e execução da solução.

4.4 Premissas e Dependências

O sistema deverá utilizar os seguintes padrões de premissas e dependências:

- Máquina do Desenvolvedor:
  - Hardware: Processador i5 ou superior (ou Apple M1/M2), 8GB de RAM, 5GB de espaço em disco disponível.
  - Software: Python 3.10+, Visual Studio Code (ou similar), Git para controle de versão.
  - Conexão: Internet estável para sincronização com o banco de dados Supabase.
- Máquina do Servidor:
  - Aplicação: Hospedada em ambiente de nuvem (Streamlit Community Cloud ou similar) com suporte a Python e renderização via WebSockets.
  - Banco de Dados: Instância gerenciada Supabase (PostgreSQL), acessível via porta 5432 ou API Rest segura.

5. Precedência e Prioridades

Crítico	Autenticação e Gestão de Usuários (RBAC), Cadastro de Pacientes e Médicos, Gestão de Agenda e Horários.
Importante	Registro de Atendimento e Faturamento, Dashboards Financeiros e Métricas
Útil	Exportação de Relatórios em PDF, Auditoria de Logs (Quem alterou o quê)

No.	Funcionalidade	Prioridade do Cliente	Entrega
1	Autenticação e Gestão de Usuários (RBAC)	Crítico	Sprint 1

2	Cadastro de Pacientes e Médicos	Crítico	Sprint 1
3	Gestão de Agenda e Horários	Crítico	Sprint 2
4	Registro de Atendimento e Faturamento	Importante	Sprint 2
5	Dashboards Financeiros e Métricas	Importante	Sprint 3
6	Exportação de Relatórios em PDF	Útil	Sprint 3
7	Auditoria de Logs (Quem alterou o quê)	Útil	Sprint 4

6. Requisitos Não-Funcionais do Produto

- **Requisitos de Sistema**
  - O Sistema deve ser leve e apresentar tempos de resposta ágeis (idealmente inferiores a 5 segundos) para a navegação entre telas e a renderização dos gráficos financeiros e de produtividade.
  - O processo não deve travar a rotina corrida do profissional durante os atendimentos.
- **Requisitos de Suportabilidade / Ambiente**
  - A aplicação operará exclusivamente em ambiente web (Nuvem).
  - O usuário final deve possuir um dispositivo (computador, tablet, smartphone) com acesso á internet e um navegador atualizado (Chrome, Edge, Safari etc.).
  - Não há necessidade de instalação de executáveis locais.
- **Requisitos de Usabilidade**
  - A interface gráfica deve ser responsiva, adaptando-se a diferentes tamanhos de tela.
  - O design deve ser minimalista e intuitivo, focando na rápida inserção de dados no formulário de consultas e na clareza dos dashboards, reduzindo a curva de aprendizado para médicos e secretárias
- **Requisitos de Confiabilidade**
  - O sistema deve garantir alta disponibilidade e a integridade absoluta dos dados.
  - A escolha do PostgreSQL assegura o funcionamento estável mesmo com acessos simultâneos, eliminando os riscos de perda e corrupção de dados comuns em planilhas locais.
- **Requisitos de Segurança**
  - O acesso à plataforma é estritamente condicionado à autenticação prévia (Login e Senha).
  - O sistema deve proteger as rotas de navegação, impedindo que usuários não logados acessem URLs diretas contendo histórico de pacientes, relatórios clínicos ou dados de faturamento.
  - As senhas dos usuários jamais trafegarão ou serão armazenadas em texto claro. O sistema adota o algoritmo de hash Bcrypt (com salt embutido), configurado para tempo de execução constante, mitigando ataques de força bruta offline.
  - A arquitetura de autenticação deve processar tentativas de login com o mesmo custo computacional e tempo de resposta, independentemente da existência ou não do usuário no banco de dados (Operação Fantasma/Dummy Hash).
  - O sistema possui um mecanismo stateless e global de bloqueio de tentativas sucessivas de login, armazenado na memória do servidor, mitigando varreduras de rede e vazamentos de sessão.
  - Em conformidade com as diretrizes de privacidade médica, qualquer sessão ociosa por mais de 30 minutos será automaticamente invalidada e purgada da memória (session_state), forçando uma nova autenticação.
  - Todas as interfaces restritas do sistema validam ativamente a credencial da sessão no primeiro milissegundo de carregamento. Tentativas de acesso direto por URL redirecionam o invasor para o painel de login.
- **Requisitos de Documentação**
  - O código-fonte (Python) deve ser estruturado em camadas lógicas (UI, Services, Database), facilitando manutenções futuras.
  - O projeto deve manter um arquivo README.md detalhado com as instruções de configuração do ambiente virtual, instalação de dependências e execução local.

7. Restrições Técnicas

7.1 Padrões Aplicáveis

- **Padrões de Codificação:** O desenvolvimento do sistema seguirá as diretrizes do PEP 8 (Python Enhancement Proposal 8). Garantindo a legibilidade, a padronização e a fácil manutenção do código-fonte escrito em Python.

- **Padrões Arquiteturais:** Adoção rigorosa do padrão de arquitetura em camadas (Apresentação/UI, Regras de Negócio/Services e Persistência/Database). Essa separação lógica é necessária para evitar o acoplamento entre os componentes visuais do Streamlit e as lógicas de acesso ao PostgreSQL.
- **Padrões de Proteção de Dados (LGPD):** Por lidar com relatórios médicos e histórico de pacientes, o projeto deve respeitar os princípios básicos da Lei Geral de Proteção de Dados. Isso exige que o armazenamento seja seguro e que o acesso às informações sensíveis seja rigorosamente restrito a usuários previamente autenticados.

7.2 Restrições Aplicáveis

- **Restrição Tecnológica (Frontend):** A construção gráfica é estritamente limitada aos componentes e comportamentos nativos oferecidos pelo framework Streamlit. Não haverá desenvolvimento de interfaces customizadas em HTML/CSS/JS puro ou frameworks como React nesta versão.
- **Restrição de Conectividade:** Como a base de dados (PostgreSQL/Supabase) e a aplicação estarão hospedados na nuvem para garantir a integridade e concorrência dos dados, o sistema não possuirá um “modo offline”. Uma conexão ativa com a internet é obrigatória para registrar consultas ou visualizar os dashboards.
- **Restrição de Plataforma:** O MVP do P4 Doctors operará exclusivamente como uma aplicação web. Não haverá desenvolvimento de aplicativos móveis nativos (arquivos .apk para Android ou .ipa para IOS). O uso em smartphones ocorrerá via navegador (responsividade).
- **Restrição de Infraestrutura e Custos:** A versão 1.0 será projetada para rodar sob as limitações dos planos gratuitos (Free Tiers) dos provedores em nuvem escolhidos. Isso impõe limites no volume máximo de armazenamento do banco de dados e na quantidade de acessos simultâneos massivos, que só serão expandidos mediante contratação de planos pagos no futuro.

8. Manual de Arquitetura

8.1 Visão de Implementação

O sistema é implementado como uma aplicação SPA (Single Page Application) lógica, utilizando o Streamlit para gerenciar o estado da sessão no servidor. O código está organizado em módulos:

- app.py: Ponto de entrada e roteamento.
- database.py: Funções de conexão e queries SQL (Supabase).
- auth.py: Lógica de encriptação (Bcrypt) e verificação de permissões.
- components/: Scripts individuais para Agenda, Faturamento e Dashboards.

8.2 Mecanismo Arquiteturais

A tabela a seguir descreve a transição entre os conceitos lógicos e as soluções técnicas adotadas para o P4 Doctors:

Mecanismo de Análise	Mecanismo de Design	Mecanismos de Implementação
Persistência de Dados	Banco de Dados Relacional (RDBMS)	Supabase (PostgreSQL): Utilização de tabelas relacionais

		com chaves UUID e integridade referencial.
Segurança e Acesso	Autenticação Baseada em Papéis (RBAC) e Criptografia	Bcrypt: Hashing de senhas com 12 rounds de salting. Supabase Auth + RLS (Row-Level Security) para isolamento de dados.
Interface com Usuário	Interface Web Reativa e Processamento no Servidor (SSR)	Streamlit Framework: Renderização dinâmica de componentes UI via WebSockets em Python.
Gestão de Relatórios	Agregação de Dados e Visualização Analítica	Pandas + Plotly: Bibliotecas integradas ao Streamlit para processamento de faturamentos e geração de dashboards.
Comunicação	API de Comunicação Segura	Biblioteca supabase-py: Cliente oficial para integração segura entre o servidor Python e a nuvem de dados via HTTPS.
Log e Auditoria	Trilhas de Auditoria e Logs de Eventos	Triggers de Banco de Dados: Funções internas do PostgreSQL que registram alterações em tabelas críticas (atendimentos/agendas).
Tratamento de Erros	Captura de Exceções e Feedback Visual	Python Try-Except + Streamlit Status: Blocos de captura de erro com alertas visuais amigáveis para o usuário na interface.

9. Dicionário de Dados

9.1 Finalidade

O Dicionário de Dados tem como objetivo definir a estrutura física de armazenamento no banco de dados relacional (PostgreSQL). A modelagem segue os princípios de Normalização, utilizando uma entidade centralizada para indivíduos (pessoas) e relacionamentos através de chaves estrangeiras (Foreign Keys) na tabela de transações (atendimentos), visando eliminar redundância e garantir a integridade analítica dos dashboards.

9.2 Tabela: atendimentos

Campo	Tipo de Dado (PostgreSQL)	Restrições	Descrição

id	UUID (ou SERIAL)	Chave Primária, Único, Not Null	Identificador único exclusivo gerado automaticamente para cada registro no banco.
data_atendimento	DATE	Not Null	Data em que a consulta ou plantão foi efetivamente realizado.
medico_id	UUID	FK, Not Null	Aponta para o id na tabela pessoas (onde tipo_perfil = 'Médico'). Identifica quem realizou o trabalho.
paciente_id	UUID	FK, Nullable	Aponta para o id na tabela pessoas (onde tipo_perfil = 'Paciente'). Fica vazio caso o registro seja referente a um Plantão geral.
local_atendimento	VARCHAR(100)	Not Null	Local físico ou virtual onde ocorreu o atendimento (ex: Hospital das Clínicas, Hospital de Base).
tipo_consulta	VARCHAR(100)	Not Null	Categoria do atendimento prestado (ex: Primeira Consulta, Retorno, Procedimento, Plantão 12h).
relatório_clinico	TEXT	Opcional	Campo de texto longo e livre para inserção de anotações médicas, evolução do paciente ou resumo do caso.
valor	NUMERIC(10,2)	Not Null, Default 0.00	Valor financeiro gerado pelo atendimento (Essencial para alimentar gráficos de faturamento).
criado_por_id	UUID	FK, Not Null	Trilha de Auditoria. Aponta para pessoas.id. Indica o usuário que estava logado e apertou o botão "Salvar"

9.3 Tabela: pessoas

Esta tabela centraliza o cadastro de todos os indivíduos que interagem com o sistema ou clínica, sejam eles os profissionais de saúde (usuários do sistema) ou os pacientes atendidos. A diferenciação ocorre através do campo tipo_perfil.

Campo	Tipo de Dado (PostgreSQL)	Restrições	Descrição
id	UUID (ou SERIAL)	Chave Primária, Único, Not Null	Identificador único do usuário no sistema.
nome_completo	VARCHAR(150)	Not Null	Nome completo do

			indivíduo.
cpf	VARCHAR(14)	Unique	Documento de identificação (Crucial para não cadastrar o mesmo paciente duas vezes).
telefone	VARCHAR(20)	Not Null	Telefone de contato do usuário.
data_cadastro	TIMESTAMPTZ	Not Null, Default NOW()	Data e hora em que a conta do usuário foi criada no sistema (Registrada de auditoria).
status	VARCHAR(20)	Not Null, Default 'Ativo'	Controle de Exclusão Lógica. Valores: 'Ativo', ou 'Inativo'. Se 'Inativo', o usuário perde acesso, mas o histórico permanece intacto.

9.4 Tabela: médicos

Tabela filha de pessoas, onde vai armazenar os médicos que estiverem presentes no sistema.

Campo	Tipo de Dado (PostgreSQL)	Restrições	Descrição
pessoa_id	UUID	Chave Primária, Único, Not Null	
crm	VARCHAR(20)	Not Null	
especialidade	TEXT	Not Null	
email	TEXT	Not Null	
senha_hash	TEXT	Not Null	

9.5 Tabela: pacientes

Tabela filha de pessoas, onde vai armazenar os pacientes que estiverem presentes no sistema.

Campo	Tipo de Dado (PostgreSQL)	Restrições	Descrição
pessoa_id	UUID	Chave Primária, Único, Not Null	
data_ nascimento	TIMESTAMPTZ	Not Null, Default NOW()	
email	TEXT	Not Null	
tipo_sanguineo	VARCHAR(3)	Not Null	
observacoes	TEXT	Not Null	

9.6 Tabela: equipe_apoio

Tabela filha de pessoas, onde vai armazenar secretárias(os) e administradores que estiverem presentes no sistema.

Campo	Tipo de Dado	Restrições	Descrição

	(PostgreSQL)		
pessoa_id	UUID	Chave Primária, Único, Not Null	
papel	TEXT	Not Null	
email	TEXT	Not Null	
senha_hash	TEXT	Not Null	

9.7 Tabela: vinculo_equipe_medico

Resolve a cardinalidade Muitos-para-Muitos (N:N), permitindo a delegação de contexto (qual recepcionista tem permissão de operar a agenda de qual médico).

Campo	Tipo de Dado (PostgreSQL)	Restrições	Descrição
equipe_id	UUID	PK (Composta), FK (equipe_apoio)	ID do profissional da secretária ou secretário.
medico_id	UUID	PK (Composta), FK (medicos)	ID do médico associado.
data_vinculo	TIMESTAMPTZ	Not Null, Default NOW()	Data em que o administrador criou a permissão.

9.8 Tabela: agendamentos

Esta tabela é responsável pelo controle de tempo e fluxo de agenda do profissional. Ela opera de forma independente da tabela de atendimentos (faturamento), garantindo que consultas futuras, cancelas ou faltas não interfiram no cálculo dos dashboards financeiros. Quando um agendamento é concluído com sucesso, ele serve de base para gerar um novo registro na tabela de atendimentos.

Campo	Tipo de Dado	Restrições	Descrição
id	UUID	PK, Unique, Not Null	Identificador único do agendamento no banco de dados.
medico_id	UUID	FK, Not Null	Aponta para o id na tabela pessoas (onde tipo_perfil='Médico´). Define a quem pertence a agenda.
paciente_id	UUID	FK, Nullable	Aponta para o id na tabela pessoas (onde tipo_perfil = 'Paciente'). Fica vazio caso o médico queira apenas bloquear a agenda para um compromisso pessoal ou plantão.
data_hora_inicio	TIMESTAMP	Not Null	Data e horário exatos em que o evento está programado para começar.

data_hora_fim	TIMESTAMP	Not Null	Data e horário exatos previstos para o término (Essencial para renderizar blocos visuais na interface de calendário).
tipo_evento	VARCHAR(100)	Not Null	Categoria do agendamento (Ex: Primeira Consulta, Retorno, Cirurgia, Bloqueio Pessoal).
status	VARCHAR(50)	Not Null, Default, ‘Agendado’	Situação atual do compromisso. Valores permitidos: ‘Agendado’, ‘Confirmado’, ‘Cancelado’, ‘Concluído', ‘Falta’.
observacoes	TEXT	Nullable	Campo livre para a secretária ou médico inserirem notas prévias (Ex: "Paciente solicitou encaixe", “Trazer exames anteriores”).
criado_por_id	UUID	FK, Not Null	Trilha de Auditoria. Aponta para pessoas.id. Identifica quem inseriu o agendamento no sistema.